

التقرير الفني النهائي لمشروع الحوسبة السحابية والحاويات

نظام إدارة المهام باستخدام Docker و REST API

اسم المقرر: حوسبة سحابية

اسم المشروع: Task Management API

أعضاء الفريق:

١. [اسم عضو الفريق الأول]

٢. [اسم عضو الفريق الثاني]

٣. [اسم عضو الفريق الثالث]

٤. [اسم عضو الفريق الرابع]

الدكتور: [اسم الدكتور]

الجامعة: [اسم الجامعة]

العام الدراسي: ٢٠٢٦

جدول المحتويات

١	 الملخص التنفيذي (Executive Summary)	2
٢	 أهداف المشروع	2
٣	 نظرة عامة على النظام	2
٤	 معمارية النظام (System Architecture)	3
٥	 REST API و CRUD	3
٦	 قاعدة البيانات PostgreSQL	3
٧	 Virtualization vs Containerization	3
٨	 Dockerfile وتحسين الطبقات	4
٩	 Docker Compose	4
١٠	 Network Architecture	5
١١	 Nginx Reverse Proxy	5
١٢	 Persistence و Storage Architecture	5
١٣	 إثبات Persistence	5
١٤	 Security Practices	5
١٥	 Failure Mode Scenarios	5
١٦	 Testing Evidence	6
١٧	 Docker Hub / Registry & Versioning	6

١٨	Reproducible Deployment.....	6
١٩	GitHub و Git	6
٢٠	Team Contributions	7
٢١	خطوات تشغيل المشروع	7
٢٢	الخاتمة	7
٢٣	Verification Status.....	7

١ الملخص التنفيذي (Executive Summary)

يقدم هذا التقرير تفصيلاً فنياً لتصميم وبناء نظام إدارة المهام (Task Management API). يهدف المشروع إلى حل مشكلة عدم توافق بيئات التشغيل (Environment Inconsistency) وتعقيد النشر (Deployment Complexity) من خلال استخدام تقنية الحاويات (Containerization) ممثلة في Docker.

يتكون النظام من واجهة برمجة تطبيقات (REST API) مبرمجة باستخدام Node.js وإطار العمل Express، والتي تتواصل مع قاعدة بيانات علائقية (Relational Database) ممثلة في PostgreSQL. ولضمان الأمان وتوجيه الطلبات بشكل مركزي، تم استخدام وكيل عكسي (Reverse Proxy) بواسطة Nginx.

استُخدم Docker Compose لربط هذه المكونات معاً لتشغيلها كبيئة واحدة معزولة، وتمت محاكاة سيناريوهات الفشل (Failure Modes) وإثبات استمرارية البيانات (Persistence). وأخيراً، تم رفع صورة النظام (Docker Image) إلى السجل العام (Docker Hub) لضمان سهولة التوزيع.

٢ أهداف المشروع

يهدف هذا المشروع إلى تحقيق عدة مخرجات تعليمية وهندسية أساسية، تشمل:

- بناء REST API: تطبيق عمليات الإنشاء والقراءة والتحديث والحذف (CRUD).
- التشغيل المعزول (Containerization): عزل كل تقنية في حاوية (Container) مستقلة.
- استخدام Docker Compose: توحيد التشغيل للخدمات المتعددة.
- وكيل عكسي (Reverse Proxy): استخدام Nginx لحماية وتوجيه الطلبات.
- العزل الشبكي (Network Isolation): حماية الخدمات الخلفية من الوصول المباشر.
- تخزين مستمر (Persistent Storage): تخزين البيانات في وحدة تخزين مسماة (Named Volume).
- اختبار الفشل (Failure Mode Testing): التحقق من قدرة النظام على التعامل مع التوقف.
- إدارة الإصدارات (Versioning): رفع الصور إلى السجل (Docker Registry).
- التشغيل المتطابق (Reproducible Deployment): القدرة على تشغيل المشروع بسهولة.

٣ نظرة عامة على النظام

يبدأ مسار الطلب من العميل ويتدرج كالتالي:

PostgreSQL → Node.js / Express API → Nginx → Client

١. Nginx: يستقبل الطلب من خارج النظام ويوجهه للداخل.
٢. Node.js API: يستقبل الطلب الموجه، يحلل البيانات، وينفذ منطق العمل.
٣. PostgreSQL: ينفذ استعلام قاعدة البيانات لحفظ أو جلب المهام.

٤ معمارية النظام (System Architecture)

يعتمد المشروع على معمارية ثلاثية الطبقات (٣-Tier Architecture) معزولة كلياً:

- Nginx: يعمل على المنفذ ٨٠ داخل الحاوية. تم ربطه بمنفذ المضيف 8080:80 (Host) ليكون نقطة الدخول الوحيدة للنظام.
- API: يعمل على المنفذ ٣٠٠٠ داخل شبكة Docker، ولا يتم نشره للمضيف. يتصل به Nginx عبر العنوان الداخلي ask-management-api:3000.
- PostgreSQL: يعمل على المنفذ ٥٤٣٢ داخل شبكة Docker، وهو معزول تماماً عن المضيف ويتصل به الـ API عبر العنوان ask-management-db:5432.

٥ REST API و CRUD

واجهة برمجة التطبيقات (REST API) هي حلقة الوصل التي تستقبل طلبات (HTTP Requests) العميل وترد عليها (HTTP Responses). تم تطبيق عمليات CRUD الأساسية كالتالي:

العملية	HTTP Method	Endpoint
إنشاء	POST	api/tasks/
قراءة	GET	api/tasks/
تعديل	PUT	api/tasks/:id/
حذف	DELETE	api/tasks/:id/

٦ قاعدة البيانات PostgreSQL

تم اختيار PostgreSQL لقوتها كقاعدة بيانات علائقية. يتكون النظام من قاعدة بيانات (Database) تحت اسم askdb تحتوي على جدول (Table) يسمى asks. كل مهمة تمثل صف (Row) يتكون من أعمدة (Columns) مثل id (كمفتاح أساسي Primary Key)، و title، و status. يتم التواصل معها عبر أوامر SQL ويرتبط بها الـ API عبر اسم خدمة Docker الداخلي: ask-management-db:5432.

٧ Virtualization vs Containerization

لإثبات كفاءة معماريتنا، نوضح الفرق بين الأجهزة الافتراضية والحاويات:

الخاصية	Virtual Machine	Container
Guest OS	يتطلب نظام تشغيل كامل لكل آلة	لا يوجد، تشارك نواة المضيف (Host Kernel)
الموارد	ثقيلة وتستهلك RAM/CPU مقتطعة	خفيفة جداً تستهلك موارد حسب الحاجة
Startup Time	دقائق	أجزاء من الثانية

الخاصية	Virtual Machine	Container (Milliseconds)
Isolation	عزل قوي (Hardware Level)	عزل على مستوى العمليات (Process Level)
Portability	صعبة النقل (ملفات بحجم جيجابايت)	سهلة النقل والتوزيع (ميجابايت)
Image	ملفات كبيرة (VHD/VMDK)	صور مبنية كطبقات خفيفة
Deployment	يتطلب إعدادات بنية تحتية معقدة	Dockerfile سريع باستخدام
Scalability	بطيء في التوسع	سريع جداً في إنشاء حاويات جديدة

يعتبر Docker الخيار الأمثل لهذا المشروع لأنه يسمح بتشغيل الطبقات الثلاث على نفس الجهاز باستهلاك موارد ضئيل وبسرعة فائقة.

٨ Dockerfile وتحسين الطبقات

إليك ملف Dockerfile المستخدم لبرمجة صورة الـ API:

```
dockerfile FROM node:24-alpine WORKDIR /usr/src/app COPY package*.json ./
RUN npm install --omit=dev COPY . . USER node EXPOSE 3000 CMD ["npm",
["start"]
```

- FROM: يعتمد على صورة مصغرة Alpine لتخفيف الحجم.
- WORKDIR: يحدد مجلد العمل داخل الحاوية.
- COPY package*.json ./: تم وضع هذا الأمر متبوعاً بـ pm install قبل نسخ باقي الكود المصدري COPY . . هذا يسمح بالاستفادة القصوى من (Layer Caching) لضمان عدم إعادة التثبيت إلا إذا تغيرت المكتبات.
- pm install --omit=dev: يثبت حزم الإنتاج فقط لتقليل الحجم وتحسين الأمان.
- USER node: يحول المستخدم من oot إلى ode لتعزيز الأمان.
- EXPOSE 3000: يوثق أن المنفذ 3000 مستخدم داخلياً، ولكنه لا يفتحه على المضيف.
- CMD: يحدد الأمر الافتراضي لتشغيل الخدمة.

٩ Docker Compose

ملف compose.yaml هو القلب النابض للنظام، حيث يعرف الخدمات:

- ask-management-db
- ask-management-api
- ask-management-nginx

استخدمنا depends_on مع healthcheck لضمان عدم عمل الـ API إلا بعد جاهزية قاعدة البيانات تماماً. وُضعت المتغيرات الأمنية في قسم environment، وتم ربط الجميع في networks. تم تخصيص ports لحاوية Nginx فقط لإبقاء الـ API و PostgreSQL مخفيين تماماً.

١٠ Network Architecture

يعتمد المشروع على شبكة من نوع Bridge Network تسمى ask-management-network. هذه الشبكة توفر خدمة نظام أسماء النطاقات الداخلي (Internal DNS) واكتشاف الخدمات (Service Discovery). بفضل هذا، يمكن لحاوية الـ API التحدث مع قاعدة البيانات بمجرد استخدام اسم الخدمة ask-management-db دون الحاجة لمعرفة عنوان الـ IP المتغير الخاص بها.

١١ Nginx Reverse Proxy

يعمل Nginx كوكيل عكسي (Reverse Proxy). مسار الطلب الفعلي هو:

ask- ↓ ask-management-api:3000 ↓ Nginx container :80 ↓ localhost:8080 ↓ Client management-db:5432

يحقق هذا فوائد هامة: * نقطة دخول واحدة للمشروع بأكمله. * إخفاء واجهة الـ API عن المضيف تماماً. * عزل شبكي كامل (Network Isolation). * تنظيم وحماية حركة المرور.

١٢ Persistence و Storage Architecture

نظام ملفات الحاوية (Container filesystem) هو نظام مؤقت يُحى بمجرد إيقاف الحاوية. لضمان بقاء البيانات، استخدمنا وحدة تخزين مسماة (Named Volume) تدعى ask-management-db-data وتم ربطها بمسار PostgreSQL: /var/lib/postgresql/data هذا يفصل دورة حياة البيانات عن دورة حياة الحاويات لضمان عدم ضياع المهام.

١٣ إثبات Persistence

تم إجراء اختبار فعلي لاستمرارية البيانات كالتالي: ١. إنشاء مهمة (Task) جديدة. ٢. تنفيذ أمر docker compose down. 3. تنفيذ أمر docker compose up -d. 4. استرجاع البيانات بأمر GET والتأكد من بقاء المهمة التي تم إنشاؤها.

تحذير: أمر docker compose down يوقف النظام بأمان، بينما أمر docker compose down -v يقوم بحذف وحدات التخزين المسماة ويمسح جميع البيانات نهائياً.

١٤ Security Practices

ركزنا على الجوانب الأمنية من خلال تطبيق الإجراءات التالية: * تشغيل API باستخدام USER node. * عدم نشر port 3000 على Host. * عدم نشر port 5432 على Host. * استخدام Nginx كنقطة الدخول الخارجية الوحيدة. * استخدام .gitignore. * استخدام .dockerignore. * عدم رفع secrets إلى GitHub. * تثبيت Production Dependencies فقط عبر --omit=dev.

١٥ Failure Mode Scenarios

تمت محاكاة انهيار النظام للتأكد من قدرته على التعامل مع الأخطاء (بناءً على الاختبارات الفعلية للمشروع):

التفسير	النتيجة	الحالة
Nginx لا يستطيع الوصول إلى	٥٠٢ Bad Gateway	توقف API

التفسير	النتيجة	الحالة
API		
API لا يستطيع تنفيذ استعلامات قاعدة البيانات	Internal Server Error ٥٠٠	توقف Database
نقطة الدخول الخارجية متوقفة	Connection Refused	توقف Nginx
API يعالج الطلب بشكل صحيح	Not Found ٤٠٤	Task ID غير موجود
API يرفض البيانات غير الصحيحة	Bad Request ٤٠٠	POST بيانات خاطئة

١٦ Testing Evidence

تم تنفيذ الاختبارات على مستويين، وفقاً للمتاح في بيئة المشروع الحقيقية:

Automated Testing ملف `pi/tests/test.js` يغطي اختبار GET للتأكد من استجابة الـ API آلياً.

Manual Runtime Testing تم التحقق من وظائف POST و PUT و DELETE واختبارات Failure Modes يدوياً أثناء التشغيل للتأكد من الموثوقية.

١٧ Docker Hub / Registry & Versioning

تم استخدام سجل حاويات (Docker Registry) في موقع Docker Hub لرفع وتوثيق الصورة الخاصة بـ API.

- Repository: hishamdiv/task-management-api
- Tags: تم استخدام ١,٠,٠ (كنسخة ثابتة) و latest (كأحدث نسخة متوفرة).
- تم تطبيق إدارة الإصدارات الدلالية (Semantic Versioning) لتنظيم التحديثات وضمان استقرار عمليات السحب (Pulling).

١٨ Reproducible Deployment

يمكن لأي مستخدم آخر تشغيل المشروع بنجاح على جهازه عبر سلسلة الأوامر التالية:

```
powershell git clone cd task-management-api docker compose up -d
```

يتحقق هذا بفضل الاعتماد على Dockerfile و compose.yaml وصور Docker Hub التي توفر بنية تحتية متطابقة في كل مرة.

١٩ Git و GitHub

تم رفع المشروع إلى مستودع التحكم بالإصدار (Repository) على GitHub. تمت إدارة التغييرات عبر لقطات (Commits) على الفرع الأساسي (Branch) المسمى main، ومن ثم دفعها إلى الخادم البعيد (Remote).

٢٠ Team Contributions

المسؤوليات	عضو الفريق
Backend Development & Containerization	[اسم عضو الفريق الأول]
Reverse Proxy & Security	[اسم عضو الفريق الثاني]
Database & Storage	[اسم عضو الفريق الثالث]
Testing & Documentation	[اسم عضو الفريق الرابع]

٢١ خطوات تشغيل المشروع

الأوامر الأساسية لتشغيل واختبار المشروع محلياً:

```
powershell cd "C:\Users\NOMAN\Projects\task-management-api" docker  
compose config docker compose up -d --build docker compose ps
```

وللتحقق من نجاح التشغيل:

```
powershell Invoke-RestMethod -Uri "http://localhost:8080/api/tasks" -Method  
GET
```

٢٢ الخاتمة

أثبت هذا المشروع نجاحه في تقديم معمارية قوية ومرنة تعتمد على حاويات Docker، حيث تم بناء واجهة تطبيقات مستقرة ومعزولة شبكياً ومحمية بواسطة وكيل عكسي. مع ضمان استمرارية البيانات وإدارة الانهيئات بطريقة سليمة، يحقق المشروع المتطلبات الأساسية للنشر السحابي الحديث.

٢٣ Verification Status

الحالة	العنصر
RUNTIME VERIFIED	Docker Compose
RUNTIME VERIFIED	API
RUNTIME VERIFIED	PostgreSQL
RUNTIME VERIFIED	Nginx
RUNTIME VERIFIED	Network Isolation
RUNTIME VERIFIED	Persistence
RUNTIME VERIFIED	CRUD
RUNTIME VERIFIED	Docker Hub
RUNTIME VERIFIED	Failure Modes
RUNTIME VERIFIED	Automated GET Test
NOT IMPLEMENTED / NOT CLAIMED	Full Automated CRUD Suite